Full Tunnel Mode or, Overriding Default Route
First, create your exit node
Enable IPv4 Forwarding on the exit node
Configure your network
Tell your local nodes to use exit node
A Linux Gotcha: rp_filter
IPv6 (optional)
IPv6 Addressing
allowGlobal and allowDefault
Set Up Gateway NDP Proxying (not always needed)
IPv6 Security
Appendix

VPN Exit Node

Full Tunnel Mode or, Overriding Default Route

Route all your Internet traffic through a ZeroTier node.

IN THIS TUTORIAL

- Create a ZeroTier Network
- Set up an `exit node` that handles all your internet traffic
- Join the `exit node` and a personal node to your ZeroTier network
- Tell your personal node to use your `exit node`

ZeroTier creates imaginary LANs. Default route override means that computers on your imaginary LAN will be routing Internet traffic through it. Just like a real LAN, your imaginary LAN is going to now need a gateway. Setup here is almost identical to what you'd do to configure a NAT gateway for a conventional wired LAN.

First, create your exit node

This is the node that will handle all of your internet traffic. It can be something as small as a $5 VPS or a Raspberry Pi.

- If you haven't already, go create a network and come back
- Install ZeroTier on it
- Join the exit node to your ZeroTier network:

```
sudo zerotier-cli join <nwid>
```

Enable IPv4 Forwarding on the exit node

On the computer acting as the VPN exit node:

```
sudo nano /etc/sysctl.conf
```

Add the line:

/etc/sysctl.conf

```
net.ipv4.ip_forward = 1
```

Tell the kernel to reload settings:

```
sudo sysctl -p
```

Check that settings were correctly applied:

```
sudo sysctl net.ipv4.ip_forward
```

If you've joined (and authorized!) your exit node to your network then you are ready to begin configuring it. Start by setting a few environment variables that we will use later:

Get the name of your ZeroTier interface, it will start with `zt`:

```
ip link show
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DE
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether 96:00:02:8d:28:c7 brd ff:ff:ff:ff:ff:ff
    altname enp1s0
3: zthnhhqofq: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 2800 qdisc fq_codel st
    link/ether 86:61:c6:fe:ed:dc brd ff:ff:ff:ff:ff:ff
```

In our case it was `zthnhhqofq`. Let's set the environment variable:

```
export ZT_IF=zthnhhqofq
```

Next, get the name of the regular WAN interface to the internet:

In our case it was `eth0`

```
export WAN_IF=eth0
```

Enable NAT and IP masquerading:

```
sudo iptables -t nat -A POSTROUTING -o $WAN_IF -j MASQUERADE
```

Allow traffic forwarding:

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCE
```

Allow traffic forwarding from the ZeroTier interface to the WAN interface:

```
sudo iptables -A FORWARD -i $ZT_IF -o $WAN_IF -j ACCEPT
```

Make `iptables` rules persist after reboot:

```
sudo apt-get install iptables-persistent
```

Save your new `iptables` rules:

```
sudo netfilter-persistent save
```

RESTART

At this point it would be a good idea to restart your exit node and verify that the routing rules have persisted.

sudo iptables-save

```
...

-A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -i zthnhhqofq -o eth0 -j ACCEPT

...

-A POSTROUTING -o eth0 -j MASQUERADE

...
```

Your exit node is complete, now we need to configure your network.

Configure your network

Now that your exit node is set up we need to configure your ZeroTier network to advertise a `Default Route` so that other nodes know that the exit node can route traffic to the internet.

Central > Network > Settings > Managed Routes

`0.0.0.0/0` via `<zerotier-ip-of-router-node>`

Tell your local nodes to use exit node

In the tray app, under each network, there is an "Allow Default" option. Check this to use your exit node.

~/.BASHRC

Try adding something like the following for simple tunnel on/off control:

```
tunnel()
{
 sudo zerotier-cli set $nwid allowDefault=1
}

notunnel()
{
 sudo zerotier-cli set $nwid allowDefault=0
}
```

A Linux Gotcha: rp_filter

Linux's networking stack is complex and almost absurdly feature-rich. This is a good thing and a bad thing. You can do almost anything with it, probably including but not limited to IP over avian carrier. But it also has a lot of weird little edge cases that can bite.

For a Linux host to route via a ZeroTier network, you may (depending on distribution) need to change a setting called rp_filter:

`sudo sysctl -w net.ipv4.conf.all.rp_filter=2`

RedHat has an article explaining the details of this. Put it in /etc/sysctl.conf to make it permanent.

Oddly enough this is not required on the gateway/router, only participating members running Linux that want to enable allowDefault.

IPv6 (optional)

With IPv6 you don't need NAT, which makes setup at this step a lot simpler! Our ip6tables looks like:

```
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
-A FORWARD -i zt+ -s 2001:19f0:6001:01a6::/64 -j ACCEPT
-A FORWARD -i eth0 -d 2001:19f0:6001:01a6::/64 -j ACCEPT
:OUTPUT ACCEPT [0:0]
COMMIT
```

Change `2001:19f0:6001:01a6::/64` to the IPv6 /64 address prefix of your gateway.

DIGITAL OCEAN

It's been a while since we checked, but Digital Ocean does not give you a real /64 and this causes some confusing issues.

IPv6 Addressing

For our configuration we enabled both IPv6 auto-assign from range and IPv6 auto-assign based on ZeroTier's RFC4193 IPv6 addressing scheme. It should also work to assign the gateway an IPv6 address from the same global prefix as everyone else and then use that for your default route, but we like the use of an NDP-emulated RFC4193 address since it makes initially reaching the gateway faster.

Our example gateway has the internal RFC4193 IPv6 address `fd80:56c2:e21c:2467:3d99:9322:c55a:1da6`, so we set that as our IPv6 default route (the IPv6 equivalent of `0.0.0.0` is `::0/0`). We also added a /80 route to the subset of our gateway's global IPv6 /64 from which we are going to assign addresses to our member devices. As with IPv4, ZeroTier will not push managed IP addresses that do not fall within an assigned local LAN route.

Our network's IPv6 routing config looks like this:

```
::0/0 via fd80:56c2:e21c:2467:3d99:9322:c55a:1da6
2001:19f0:6001:01a6:00ff::/80 (LAN)
```

Then we will want to configure ZeroTier Central's controller to automatically assign public IPv6 addresses to our devices by defining an IPv6 auto-assign range.

Ours looks like:

```
Auto-Assign from Range
2001:19f0:6001:01a6:00ff:0000:0000:0000
2001:19f0:6001:01a6:00ff:0fff:ffff:ffff
```

allowGlobal and allowDefault

For IPv6 you must enable both `allowGlobal` and `allowDefault` on your member devices, or at least on those that you want to participate. Allowing default route override is required to allow the override of `::0/0`, while `allowGlobal` is required to allow your network's controller to assign global IPv6 addresses. Without `allowGlobal` ZeroTier networks are only allowed to assign addresses and routes within private IP space.

Set Up Gateway NDP Proxying (not always needed)

At this point you should stop and make sure that (1) your devices are getting assigned IPv6 addresses, (2) your member device(s) can ping your gateway's private RFC4193 IPv6 address, and (3) member devices can also ping your gateway's global external IPv6 address. The latter verifies that the gateway is in fact forwarding IPv6 traffic.

If the above tests work, try pinging a global IPv6 address like ping6 ipv6.google.com. If that doesn't work you probably need to proxy NDP requests for the hosts behind your gateway.

While providers like will happily hand you a wildcard /64, they do not necessarily set a route to your entire IPv6 /64 in their own routers. They probably do this to prevent routing table bloat on their side. Instead they rely upon IPv6 neighbor lookup (NDP) to find out what IP address(es) your VM has assigned to itself.

The problem is that now your VM is a router and wants to get traffic for a whole bunch of IPs in its /64, not just the one it's got assigned to its primary network interface. To do this, it's going to have to answer IPv6 NDP queries for those IPs.

The Linux kernel can't do this, or at least can't for every IP in your /64 without manually specifying each. Luckily there is a nice little project on GitHub called ndppd that can do this for you.

Your distribution may have a package.

```
apt install ndppd
```

Now you'll want to copy ndppd.conf-dist from the ndppd source tree to /etc/ndppd.conf and edit it. We just had to edit the prefix under their example rule entry:

```
rule 2001:19f0:6001:01a6:00ff:0000:0000:0000/80 {
```

That tells ndppd to answer NDP requests for the entire /80 from which we'll be assigning IPv6 addresses to our devices. Obviously you will need to change that IP prefix to your own.

There is a handy program to help you with ip subnetting: `apt install ipcalc`

The rule should use the `static` or `iface` option. We saw some flakiness with `auto`.

Then you'll want to start ndppd and tell it to start on boot.

```
systemctl start ndppd
systemctl enable ndppd
```

Once ndppd is running try ping6 ipv6.google.com again from one of your devices. For us it worked right away!

Congratulations! You now have a global IPv6 address for every device on your virtual network.

```
curl -4 ifconfig.co
curl -6 ifconfig.co
```

Should return your ZeroTier gateway addresses.

IPv6 Security

This configuration gives every device on your ZeroTier network a real globally reachable IPv6 address. This is wonderful but also possibly a little bit dangerous.

If you don't want everything to be wide open like that you can use ip6tables rules to implement a stateful firewall that allows all outbound IPv6 traffic but limits inbound traffic to packets that are replies to outbound ones.

This can be done by using an alternative configuration like:

```
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
-A FORWARD -i zt+ -s 2001:19f0:6001:01a6::/64 -j ACCEPT
-A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
:OUTPUT ACCEPT [0:0]
COMMIT
```

This allows outgoing IPv6 from the ZeroTier side but only allows incoming IPv6 if it's related to an existing connection or association.

Much more sophisticated configurations are possible. For example you could insert specific rules allowing specific kinds of inbound traffic like ssh (22), http (80), or https (443) by adding these rules ahead of the stateful one.

Appendix

SNAT

If you have a public static IP, you can use SNAT instead of MASQUERADE.

```
*nat
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A POSTROUTING -o eth0 -s 10.6.4.0/22 -j SNAT --to-source <public-ip>
COMMIT
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
-A FORWARD -i zt+ -s 10.6.4.0/22 -d 0.0.0.0/0 -j ACCEPT
-A FORWARD -i eth0 -s 0.0.0.0/0 -d 10.6.4.0/0 -j ACCEPT
:OUTPUT ACCEPT [0:0]
COMMIT
```

The iptables manual explains:

"This target (masquerade) is only valid in the nat table, in the POSTROUTING chain. It should only be used with dynamically assigned IP (dialup) connections: if you have a static IP address, you should use the SNAT target. Masquerading is equivalent to specifying a mapping to the IP address of the interface the packet is going out, but also has the effect that connections are forgotten when the interface goes down. This is the correct behavior when the next dialup is unlikely to have the same interface address (and hence any established connections are lost anyway)."

FreeBSD

The allowDefault=1 setting on FreeBSD clients can't work. See this github issue for a work-around https://github.com/zerotier/ZeroTierOne/issues/580.

We've disable the Allow Default setting on FreeBSD starting on versions newer than 1.10.6 until we can find a solution.

Tags: tutorial example

Copyright © 2025 ZeroTier, Inc. All rights reserved.